

Multiview Experiment Log

"KITTI 제출용 논문 baseline을 확보하고 싶다"는 요청에서 시작해, Car 클래스 멀티뷰 확장, 자동 학습 큐 구성, Cyclist 방향 예측 붕괴 발견과 원인 규명까지 이어진 전체 과정을 시간순으로 남긴 기록.

리포지토리: `DILAB-pointpillars` (main) + `DILAB-pointpillars-paper-baseline` (worktree) · 총 10개 학습 job, GPU 4장(RTX A5000) 순차 사용

목차

- | | |
|---------------------------|-------------------------|
| 00 최초 요청 | 07 검증 후 큐 자동화 |
| 01 "baseline이 뭔지"부터 다시 정의 | 08 전체 결과 취합 |
| 02 실행 환경 분리(worktree) | 09 이상 현상: Cyclist 방향 붕괴 |
| 03 GPU 오픈과 정정 | 10 근본 원인 조사(2단계 진단) |
| 04 범위 확장: Car도 필요 | 11 시드 미고정 확인 |
| 05 스크립트 일반화(Car 태스크 포팅) | 12 현재 상태: 정말 개선됐나 |
| 06 지금 모델 구조가 정확히 뭔지 | 13 남은 작업 |

00 최초 요청

배경

"우리 PointPillars 모델 test 데이터 있나, KITTI에 제출해서 성능 좀 받아보고 싶다 — 내가 개선한 거 말고, 논문을 구현한 거"로 시작. 즉 목표는 처음부터 **연구용 개선 코드가 아니라 순수 nuTonomy PointPillars 구현의 KITTI 성능이었음**.

01 "baseline이 뭔지"부터 다시 정의해야 했음

조사

문제

처음엔 단순히 `git stash` 로 uncommitted 변경사항만 걷어내면 될 줄 알았음. 그런데 `git log` 를 까보니:

```
449c7c0 2019-05-06, Alex Lang — nuTonomy의 마지막 원본 커밋
9ed03a3 2026-09-07, bohyeon35 — "code" (멀티뷰 아키텍처 전체가 이미 커밋됨)
```

멀티뷰 개선사항 상당 부분이 **이미 커밋으로 들어가 있었음** — uncommitted diff만 걷어내는 걸로는 순수 baseline이 안 나옴. 그래서 449c7c0로 완전히 되돌리는 걸 시도하려 했는데, 두 커밋 사이 diff를 보니 `second/core/box_ops_cc.py`, `non_max_suppression/nms_gpu.py` 등에 "**이 환경에 C++ 컴파일러가 없어서 만든 순수 파이썬 대체 구현**", "**이 numba 버전에서 device 함수 max/min이 깨져서 우회**" 같은 호환성 패치가 섞여 있었음. 이것들은 "개선"이 아니라 **이 서버에서 아예 돌아가게 만드는 필수 패치**라, 449c7c0로 되돌리면 학습 자체가 실패할 위험이 있었음.

최종 판단: 코드를 뜯어보니 멀티뷰 관련 함수들이 전부 `view='xy'` (기본값)일 때 "baseline PointPillars와 동일하게 동작"하도록 설계돼 있었음 (`second/core/view_transform.py` 주석에 명시). 그래서 449c7c0로 되돌리는 대신, **현재 코드 + 기본값 view='xy'**를 baseline으로 채택 — 호환성 패치는 유지하면서 개선 로직은 안 타는 경로.

02 실행 환경 분리 — git worktree

변경

다만 메인 레포는 계속 유저의 우선 연구(멀티뷰 실험, 나중엔 Car 멀티뷰까지)가 진행 중이라 코드가 계속 바뀜. baseline 학습이 그 변화에 영향받지 않도록:

```
git worktree add -b paper-baseline ../DILAB-pointpillars-paper-baseline 9ed03a3
```

커밋 9ed03a3 그대로(유저의 uncommitted 개선사항 없이) 별도 디렉토리로 체크아웃. `.proto` 설정 파일의 로컬 경로(데이터 위치 등)만 메인 레포와 동일하게 재적용. 이후 **Car/Ped baseline**은 전부 이 워크트리에 서, **멀티뷰 실험**은 전부 메인 레포에서 돌아가는 구조로 완전히 분리.

03 GPU 오판과 정정

실수

정정

baseline 학습을 시작하며 `nvidia-smi` 스냅샷만 보고 GPU1이 한가하다고 판단해 그 위에 학습을 엮었음. 그런데 실제로는 **GPU 4장 전부 유저 본인의 기존 Ped/Cyclist 멀티뷰 실험(xy_only/xz/yz/xz+yz) 4개가 이미 5시간 넘게 돌고 있던 상태였음** — 순간 사용률이 낮게 찍힌 걸 "비어있다"로 오판.

유저가 "그 GPU 4개 지금 뭐 하고 있는 거임?"이라 물어봐서 발견. 즉시 프로세스를 죽였는데, **DataLoader worker** 자식 프로세스가 부모 **kill** 후에도 **orphan**으로 살아남아 GPU 메모리를 계속 점유하는 걸 확인 — `pkill -9` 로 남은 워커까지 강제 정리.

이후 방침을 "4개 중 아무거나 하나 끝나면 그 자리에 넣어라"로 바뀌서, 기존 연구를 방해하지 않게 재설계.

04 범위 확장 — "Car도 해야지" → "멀티뷰에도 Car를 해야지"

범위 변경

대화 도중 요구사항이 두 번 확장됨:

- 처음엔 "Pedestrian/Cyclist만 우선 제출" → 이후 "아니 차도 해야지"로 Car baseline 추가
- Car baseline 학습을 큐에 넣은 뒤 "아니아니 멀티뷰에도 car를 해야하는데" → Car **멀티뷰 ablation**(xy_only/xz/yz/xz+yz 4종)까지 필요해짐

문제는 `multiview_experiment.py` 가 애초에 `CLASS_NAMES = ["Cyclist", "Pedestrian"]` 로 **Ped/Cyclist 전용으로 하드코딩**되어 있었다는 것 — Car를 돌리려면 스크립트 자체를 고쳐야 했음.

05 multiview_experiment.py 일반화 — Car 태스크 포팅

코드 변경

모듈 상수로 박혀있던 값들을 `TASKS = {'ped_cycle': {...}, 'car': {...}}` 딕셔너리로 재구성. Car용 값은 `configs/pointpillars/car/xyres_16.proto` 에서 그대로 가져옴 (point_cloud_range, voxel_size, anchor size, RPN stride, DB 샘플링 파라미터 등).

가장 까다로웠던 부분은 **보조 브랜치(xz/yz)의 Z축 패딩 계산**이었음. RPN이 3단 backbone(stride 곱)을 거쳐 다시 upsample-concat 되는 구조라, 보조 브랜치의 Z 방향 필라 개수가 **RPN 전체 다운샘플 배율의 배수**여야 shape이 맞음:

태스크	RPN LAYER_STRIDES	다운샘플 배율	Z 패딩 결과
ped_cycle	[1, 2, 2]	×4	-2.6 ~ 0.6m → 20칸(4의 배수)
car	[2, 2, 2]	×8	-3.56 ~ 1.56m → 32칸(8의 배수)

이 값을 잘못 계산하면 `ChannelFoldFusion` 에서 xy 브랜치와 xz/yz 브랜치의 공유 축(X 또는 Y) 길이가 안 맞아 concat이 실패함. GPU 시간을 낭비하기 전에 먼저 **10-step smoke test**(`--task=car --branches=xz,yz --steps=10`)로 shape 정합성부터 확인 — 로그에 `Z'=16, matches xy width X=216: True` 식으로 검증 로그가 찍히도록 이미 설계돼 있어서 바로 확인됨.

06 지금 모델 구조가 정확히 뭔지

아키텍처

여기서 나온 모든 숫자가 어떤 네트워크에서 나왔는지 명확히 해두면: 이번 세션은 서로 다른 두 네트워크 클래스를 돌렸음 — 둘 다 `second/pytorch/models/voxelnet.py` 에 있고, 저수준 부품 (`second/pytorch/models/pointpillars.py` 의 `PillarFeatureNet` , `PointPillarsScatter`) 은 공유.

① 논문 baseline — **VoxelNet** (단일 브랜치, 원본 구조)

포인트 → `PillarFeatureNet` (필라별 point-wise feature) → `PointPillarsScatter` (2D pseudo-image로 흠뿌림) → 3단 backbone+deconv(RPN 클래스, backbone과 head가 한 몸) → `conv_cls` / `conv_box` / `conv_dir_cls` 로 클래스·박스·방향 예측. XY 평면 하나만 씬. 이게 원래 **PointPillars** 논문 그대로의 구조.

② 멀티뷰 — **MultiViewVoxelNet** (다중 브랜치 + 채널 접기 융합)

①의 backbone을 `RPNBackbone` (순수 conv 부분)과 `DetectionHead` (`conv_cls/box/dir_cls`)로 분리한 뒤, **branch마다(xy는 필수, xz/yz는 선택) `PillarFeatureNet+PointPillarsScatter+RPNBackbone`을 독립적으로** 하나씩 만듦(interpolation 없이 각자 native 해상도 유지). 그 결과를 `ChannelFoldFusion` 이 다 음처럼 합침:

- xz 브랜치: 자기 축소된 Z'를 채널로 접어서 xy 해상도로 broadcast
- yz 브랜치: 마찬가지로 Z'를 채널로 접어서 broadcast
- xy·xz'·yz' 전부 channel-concat 후 1×1 conv로 `fused_channels=128` 로 축소

이 융합된 feature 하나를 공유 **DetectionHead** 하나에 통과시켜 최종 예측 — anchor/box coder/target assigner는 전부 표준 XY 7차원(x,y,z,w,l,h,r) 그대로라, xz/yz는 "박스를 직접 예측"하는 게 아니라 xy 예측을 보조하는 **feature 공급원** 역할만 함.

공통 하이퍼파라미터: `RPN layer_nums=[3,5,5]` , `num_filters=[64,128,256]` , `upsample_strides=[1,2,4]` · Adam, `lr=2e-4` (decay 27,840 step마다 ×0.8) · `cls=SigmoidFocal( $\alpha$ 0.25, $\gamma$ 2.0)` + `loc=SmoothL1( $\sigma$ 3.0)`, 가중치 각 1.0/2.0, 방향 0.2 · 배치사이즈 2 · 296,960 step(160 epoch).

Car vs Ped/Cyclist 태스크별 차이:

	CAR	PED/CYCLIST
point_cloud_range	[0, -39.68, -3, 69.12, 39.68, 1]	[0, -19.84, -2.5, 47.36, 19.84, 0.5]
voxel_size (z)	0.16 / 0.16 / 4	0.16 / 0.16 / 3
XY grid	432×496	296×248
RPN layer_strides	[2,2,2] (out_size_factor=2)	[1,2,2] (out_size_factor=1)

anchor 종류	1개 (1.6×3.9×1.56)	2개 (Cyclist 0.6×1.76×1.73, Pedestrian 0.6×0.8×1.73)
class 수	1	2 (동시 예측)

참고: `second/multiview_experiment.py` 의 TASKS 디렉터리, `second/pytorch/models/voxelnet.py` 의 MultiViewVoxelNet / ChannelFoldFusion / DetectionHead 클래스, 상세 결과는 `pillar_axis_ablation_report.pdf` 참고.

07 검증 후 큐 자동화

자동화

smoke test 통과 후, 아래 6개 작업을 순서대로 큐잉 — 기존 4개 프로세스의 PID를 감시하다가 하나라도 죽으면(=완료) 그 GPU에 큐의 다음 항목을 자동 실행하는 monitor 스크립트를 작성해서 백그라운드로 돌림:

1. car_mv_xy_only
2. car_mv_xz
3. car_mv_yz
4. car_mv_xz_yz
5. car_paper_baseline (worktree)
6. pedcycle_paper_baseline (worktree)

기존 4개 + 신규 6개, 총 **10개 job을 GPU 4장으로 순차 처리**. 각 job은 296,960 step(160 epoch) 완주 기준이라 전체 파이프라인이 끝나기까지 하루 이상 소요됨. 진행 중 GPU 온도(70°C 제한) 감시 모니터도 별도로 유지.

08 전체 결과 취합

결과

10개 job이 전부 완료된 뒤(중간중간 완료되는 대로 평가도 같이 돌림) 나온 최종 3D AP(Easy/Moderate/Hard):

구분	EASY	MODERATE	HARD
Car 논문 baseline	84.16	75.14	69.14
Car xy_only	80.36	75.89	68.92
Car xy+xz	79.96	70.08	68.60
Car xy+yz	81.19	70.29	67.29

Car xy+xz+yz	80.66	70.05	68.90
Cyclist 논문 baseline	82.00	60.78	56.87
Cyclist xy_only	76.17	50.81	50.01
Cyclist xy+xz	70.17	49.48	48.95
Cyclist xy+yz	75.80	53.21	49.89
Cyclist xy+xz+yz	66.60	44.01	43.35
Pedestrian 논문 baseline	66.06	60.41	55.25
Pedestrian xy_only	63.43	56.19	48.40
Pedestrian xy+xz	57.77	56.74	48.83
Pedestrian xy+yz	66.62	60.08	53.65
Pedestrian xy+xz+yz	56.23	41.65	40.87

(Car는 IoU 0.7, Cyclist/Pedestrian은 IoU 0.5 KITTI 공식 기준. 자세한 차트는 별도 리포트 [pillar_axis_ablation_report.pdf](#) 참고.)

09 이상 현상 발견 — Cyclist 방향 예측 붕괴

이상 현상

결과를 뜯어보다가 xy+xz+yz 의 Cyclist에서 **aos(방향 정확도)**만 **3.76 / 6.32 / 6.27**로 다른 지표 (bbox/3D AP 44~68)와 완전히 동떨어진 걸 발견. 나머지 xy_only/xz/yz는 전부 aos ≈ bbox AP로 정상.

10 근본 원인 조사 — 체크포인트를 직접 열어서 진단

근본원인 조사

먼저 코드 리뷰: `ChannelFoldFusion`, `DetectionHead`, `predict()` 의 NMS/방향 디코딩 로직을 직접 읽어봤는데, **전부 클래스 수·브랜치 수와 무관한 단일 경로**라 인덱싱이 꼬일 지점을 못 찾음. 그래서 실제 체크포인트를 로드해서 두 단계로 직접 측정하는 진단 스크립트를 작성:

단계	측정 방법	CYCLIST 결과	PEDESTRIAN 결과
1단계	학습-타겟 기준 dir 정확도 (positive anchor)	91.0% (256개)	62.8% (355개)
2단계	최종 검출 vs GT heading 오차 (실제 aos가 보는 값)	78.9%가 ~180° 반전	-

1단계만 보면 오히려 Cyclist가 더 정확했는데, 2단계(실제 최종 검출)에서는 **거의 항상 방향이 180도 뒤집혀서 나옴** — 각도 오차 히스토그램이 정확히 $\pm 180^\circ$ 근처에만 몰려있고 중간이 텅 빈, 노이즈가 아닌 체계적 반전 패

턴.

결론: 소프트웨어 버그가 아니라 학습 수렴 실패로 판단. Cyclist 박스(0.6×1.76 , 가늘고 긴 직사각형)는 $0^\circ/180^\circ$ 헤딩이 IoU 관점에서 거의 구분이 안 되고, `direction_loss_weight=0.2` 로 상대적으로 약하게 걸려있어서, 데이터가 적은(734개) 이 클래스가 "위치는 맞지만 방향은 반대"인 local minimum에 빠지기 쉬운 조건이었던 것으로 판단.

11 시드 미고정 확인

확인

방향 붕괴가 "이 run에서 우연히 걸린 local minimum"이라는 가설을 뒷받침하는 사실 하나 더 확인: 이 코드베이스는 어디에도 랜덤 시드가 고정돼 있지 않음.

```
# second/pytorch/train.py:87-90
def _worker_init_fn(worker_id):
    time_seed = np.array(time.time(), dtype=np.int32)
    np.random.seed(time_seed + worker_id)
```

`torch.manual_seed()` 는 리포 전체에 없고, `multiview_experiment.py` 는 이 worker seed 함수조차 안 씀. 즉 augmentation, 셔플 순서, 가중치 초기화가 실행할 때마다 전부 달라짐 — 재현성 검증이 필요하면 시드부터 고정해야 함.

12 현재 상태 — 모델이 정말 "개선"됐나

현황 평가

솔직하게 정리하면: 이번 세션에서 뚜렷하게 "개선됐다"고 말할 수 있는 조합은 제한적입니다.

- **xy+yz**만 Pedestrian·Cyclist에서 xy_only 대비 안정적으로 개선 (Pedestrian Moderate +3.9pt, Cyclist Moderate +2.4pt). 다만 **Car**에서는 오히려 **xy_only**보다 낮음 — 클래스에 따라 효과 방향이 다름.
- **xy+xz**는 세 클래스 어디서도 뚜렷한 이득이 없음.
- **xy+xz+yz**(3-브랜치)는 Ped/Cyclist에서 가장 낮은 성능 + 방향 버그까지 동반 — 현재 fusion 설계로는 브랜치를 늘릴수록 손해.
- 논문 baseline(순수 구현) 수치는 원 논문 리포트와 근접해서, **baseline 자체의 재현성은 확인됨**.

즉 이번 세션의 실질적 성과는 "모델을 개선했다"보다는 "어떤 브랜치 조합이 언제 도움이 되고 언제 해가 되는지, 그리고 그 안의 버그가 정확히 어디서 발생하는지를 데이터와 코드 양쪽에서 규명한 것"에 가깝습니다.

○ 13 남은 작업

TODO

- KITTI test 서버 실제 제출용 결과 파일 생성 (baseline 체크포인트로 test set 추론 → 제출 포맷 zip) — 아직 미착수
- 시드 고정 후 $xy+xz+yz$ 재학습으로 방향 붕괴 재현 여부 검증
- 재현되면 `direction_loss_weight` 상향 또는 heading-symmetry-aware loss 적용 실험
- $xy+yz$ 가 왜 Car에서만 역효과인지 별도 분석 필요

함께 보면 좋은 자료: `second/logs/reports/pillar_axis_ablation_report.pdf` (차트 중심 결과 대시보드) ·

`/home/bohyeon/.claude/plans/tidy-purring-sutton.md` (멀티뷰 아키텍처 설계 문서). 이 로그의 시각/스텝 수치는 학습 로그(`second/logs/*.log`)와 진단 스크립트 실행 결과를 근거로 작성했습니다.